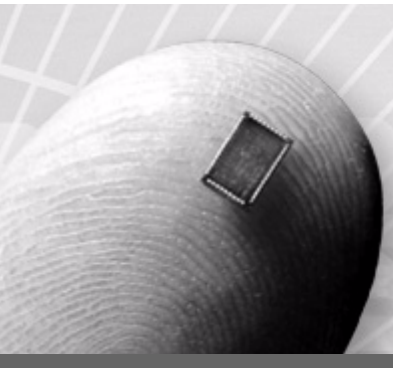
ON Semiconductor®



CFX Communication Protocols Manual

M-20743-004

ON Semiconductor

September 2011

This document may not be redistributed or transmitted in any form.

Copyright © 2011 Semiconductor Components Industries, LLC d/b/a ON Semiconductor.

ON Semiconductor and the ON logo are registered trademarks of Semiconductor Components Industries, LLC (ON Semiconductor). All other brands, product names and company names mentioned herein may be trademarks or registered trademarks of their respective holders.

ON Semiconductor reserves the right to make changes without further notice to any products herein. ON Semiconductor makes no warranty, representation or guarantee regarding its products including suitability for any particular purpose, nor does ON Semiconductor assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in ON Semiconductor data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. ON Semiconductor does not convey any license under its patent rights nor the rights of others. ON Semiconductor products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the ON Semiconductor product could create a situation where personal injury or death may occur, unless otherwise expressly agreed to in writing by ON Semiconductor and BUYER. Should Buyer purchase or use ON Semiconductor products for any such unintended or unauthorized application, Buyer shall indemnify and hold ON Semiconductor and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that ON Semiconductor was negligent regarding the design or manufacture of the part. ON Semiconductor is an Equal Opportunity/Affirmative Action Employer. Literature is subject to all applicable copyright laws and is not for resale in any manner.

To maintain the quality of our publications, we request your comments on the accuracy, clarity, organization, and value of this document.

Address correspondence to:

Americas and Asia
ON Semiconductor
200-611 Kumpf Drive
Waterloo, Ontario, Canada N2V 1K8
Tel: +1 519 884 9696

Europe, Middle East and Africa
ON Semiconductor
Champs-Montants 12a
2074 Marin, Switzerland
Tel: +41 32 755 74 30

Website
www.onsemi.com

E-mail
dsp.support@onsemi.com

TABLE OF CONTENTS

1. Introduction 1

1.1 Purpose . . . 1
1.2 Intended Audience . . . 1
1.3 Conventions . . . 1
1.4 Manual Organization . . . 1
1.5 Further Reading . . . 2

2. Algorithm Protection 3

2.1 Overview . . . 3
2.2 Restricted and Unrestricted Access Modes . . . 3
2.2.1 Direct Configuration . . . 4
2.2.2 JumpROM Configuration . . . 4
2.3 Protected Resources . . . 4
2.3.1 Protected Resources in Non-Volatile Storage . . . 4
2.3.2 Protected Resources in Memory . . . 5
2.4 File System Security Support . . . 5

3. JumpROM Function Set 7

3.1 Background . . . 7
3.2 JumpROM NOP . . . 7
3.3 JumpROM GetVersion . . . 7
3.4 JumpROM Reset . . . 8
3.5 JumpROM Unlock . . . 8
3.6 JumpROM Wipe . . . 8
3.7 JumpROM GetError . . . 8
3.8 JumpROM BootApp . . . 9
3.9 JumpROM Lock . . . 9

4. Debug Port Protocol 11

4.1 Introduction . . . 11

4.2 Basic Protocol Information 11
4.2.1 Debug Port Addressing 11
4.2.2 Acknowledgements 12
4.2.3 Command Format 12
4.2.4 Response Format 13
4.2.5 Modes 13
4.3 Debug Port Protocol Commands and Responses 14
4.3.1 Overview 14
4.3.2 Initialize Transmit State 15
4.3.3 Debug Port State and Information Commands and Responses . . . 15
4.3.3.1 NOP Command 15
4.3.3.2 Read Status Command 16
4.3.3.3 Status Transmit State 16
4.3.3.4 Write Address Command 17
4.3.3.5 Acquire Address Command 18
4.3.3.6 Reset Debug Port Command 18
4.3.3.7 Set Monitor Reset Bit Command 18
4.3.3.8 Read System ID Command 18
4.3.3.9 SysID Transmit State 19
4.3.3.10 Read and Reset CRC Command 19
4.3.3.11 CRC Transmit State 20
4.3.3.12 Reset and Start Cycle Counter Command 20
4.3.3.13 Get Cycle Count Command 20
4.3.3.14 Cycle Count Transmit State 21
4.3.4 Memory Access Commands and Responses 21
4.3.4.1 Transfer Mode 21
4.3.4.2 Read Memory Command 22
4.3.4.3 Memory Transmit State 23
4.3.4.4 Write Memory Command 23
4.3.5 Register Access Commands and Responses 23
4.3.5.1 Read Normal Registers Command 24
4.3.5.2 Normal Registers Transmit State 25
4.3.5.3 Write Normal Registers Command 26
4.3.5.4 Read Special Registers Command 26
4.3.5.5 Special Registers Transmit State 27
4.3.5.6 Write Special Registers Command 27

4.3.5.7 Read Debug Port Register Command 28
4.3.5.8 Debug Port Register Transmit State 28
4.3.5.9 Write Debug Port Register Command 28
4.3.6 Run Control Commands 29
4.3.6.1 Start Core Command 29
4.3.6.2 Stop Core Command 29
4.3.6.3 Single-Step Command 29
4.3.6.4 Single-Step With Interrupts Command 29
4.3.6.5 Execute Instruction Command 30
4.3.6.6 JumpROM Command 30
4.3.7 Breakpoint Commands and Responses 30
4.3.7.1 Set Hardware Breakpoints Command 30
4.3.7.2 Write Breakpoint Config Command 31
4.3.7.3 Read Breakpoints Command 32
4.3.7.4 Breakpoints Transmit State 32

A. JumpROM Function Quick Reference *33*

B. Debug Port Protocol Quick Reference *35*

This page left blank for printing purposes.

1 Introduction

1.1 Purpose

This manual provides information regarding the communication protocols for application developers working with chips based on the CFX Digital Signal Processor (DSP). This manual discusses the built-in method of communicating with these chips: the debug port protocol.

This manual further explains the system algorithm protection feature, and how this feature affects the debug port protocol in its efforts to protect potentially sensitive intellectual property (IP) in the form of application code and data.

The *CFX Communication Protocols Manual* is part of the Evaluation and Development Kit (EDK).

1.2 Intended Audience

This manual is intended for engineers and anyone else responsible for implementing a communications infrastructure communicating with chips based on the CFX DSP. This manual assumes that you know how to use the interfaces, have a general familiarity with communication protocols, and are familiar with the chips. If you are going to use the debug port protocol, it is helpful to have some familiarity with the CFX DSP instruction set.

1.3 Conventions

This manual uses the following conventions:

- Any configuration registers, macros and CFX instructions are shown in a `special font`.
- Debug port commands are in **bold** typeface.
- Debug transmit states are in *italic* typeface.
- In general, numbers are presented in decimal notation. In cases where hexadecimal or binary notation is more convenient, these numbers are identified by the prefixes “0x” and “0b” respectively. For example, the decimal number 123456 can also be represented as 0x1E240 or 0b11110001001000000.

1.4 Manual Organization

The *CFX Communication Protocols Manual* contains the following chapters and appendices:

- **Chapter 1: Introduction**, describes the purpose of the manual, the intended audience, conventions used in this manual, outlines how the book is organized, and gives a list of suggested reading for more information.
- **Chapter 2: Algorithm Protection** provides an overview of the algorithm protection features and how the various components of the algorithm protection scheme work together.
- **Chapter 3: JumpROM Function Set** contains a detailed description of the JumpROM functions, including the preconditions for commands and their relation to algorithm protection.

- **Chapter 4: Debug Port Protocol** contains a detailed description of the built-in debug port protocol, including system state information and details on the functionality of each of the commands in the protocol.
- **Appendix A: JumpROM Function Quick Reference** provides a quick reference to the JumpROM functions.
- **Appendix B: Debug Port Protocol Quick Reference** provides a quick reference to the commands available through the debug port protocol.

1.5 FURTHER READING

For more information, refer to the following documents:

- *Integrated Development Environment User's Guide*
- *CFX Toolchain Reference*
- *Hardware Reference Manual*
- *Firmware Reference Manual*
- *Evaluation and Development Board Manual*
- *CFX DSP Architecture Manual*
- *The I2C-Bus Specification*, Philips Semiconductor, document order number 9398 393 40011

2 Algorithm Protection

2.1 Overview

The algorithm protection scheme provides a mechanism for protecting intellectual property in the form of application code and data stored in memory and non-volatile memory resources.

Upon startup or following a reset, the algorithm protection scheme is automatically set to protect system resources, restricting all memory access through the debug port. Additionally, through support for the use of file system encryption in the non-volatile storage area, the application code and data that is stored in the non-volatile storage area can be protected even when the device package does not physically prevent direct access to the non-volatile storage medium.

2.2 Restricted and Unrestricted Access Modes

The chips operate in one of two different access modes: restricted access mode and unrestricted access mode.

- When operating in restricted access mode, external access to application code or data executing on the device is not available through the debug port interface as certain debug port commands are disabled. A summary of debug port commands and restrictions based on access mode can be found in Appendix B, “Debug Port Protocol Quick Reference” on page 35.
- When operating in unrestricted access mode, external access to application code or data executing on the device is available as all debug port interface commands are available.

To query the current debug port status, check the security mode bit from the debug port status word. For details on querying and interpreting the debug port status, see Section 4.3.3, “Debug Port State and Information Commands and Responses” on page 15.

To change the access mode, use one of the following methods:

- The application running on the CFX DSP core can configure the mode directly, as described in Section 2.2.1, “Direct Configuration”.
- Through the debug port JumpROM commands, the debug host can configure the mode externally, as described in Section 2.2.2, “JumpROM Configuration”.
- If an application boots the device through the debug port instead of a non-volatile storage device attached to the SPI interface, the boot ROM can set the access mode to unrestricted as follows:
  - Specify the boot address as 0x0000 or 0xFFFF.
  - Set the key area to all zeros or all ones.

Note: If no EEPROM is attached to the device, you can pull the SPI_SERI pin to ground or up to the same level as VDDO_SPI to ensure that all zeros or all ones are read from the device for both the boot address and key area.

IMPORTANT: If the application running on the CFX DSP core implements a data transfer mechanism using other external interfaces, such as the general purpose I²C interface, the application is responsible for limiting access to application code or data to protect intellectual property.

2.2.1 Direct Configuration

The access mode can be configured using the DEBUG_CFG_ACCESS_MODE bit in the D_DEBUG_CFG register. The Sys_DebugPort_Set_AccessMode macro provides a standard method of updating the current access mode.

The contents of memory or non-volatile storage do not need to be cleared when directly switching between access modes.

2.2.2 JumpROM Configuration

The system access mode can be reconfigured by an external device through debug port commands using the JumpROM command set.

To select restricted access mode, execute a JumpROM lock function. No restrictions are placed on the contents of memory or non-volatile storage when executing this function. For more details on the JumpROM lock function, see Section 3.9, “JumpROM Lock” on page 9.

To select unrestricted access mode, use JumpROM functions to clear the protected resources in non-volatile storage and memory, and update the access mode:

1. Run the JumpROM wipe function to clear the protected resources in non-volatile storage. Details of protected resources in non-volatile storage are provided in Section 2.3.1, “Protected Resources in Non-Volatile Storage”. For more details on the JumpROM wipe function, see Section 3.6, “JumpROM Wipe” on page 8.
2. Run the JumpROM unlock function to clear the protected resources in memory and select unrestricted access mode. Details of protected resources in memory are provided in Section 2.3.2, “Protected Resources in Memory”. For more details on the JumpROM unlock function, see Section 3.5, “JumpROM Unlock” on page 8.

   Note: You can use the JumpROM unlock function if non-volatile storage is not attached to the device, or if the non-volatile storage has already been cleared. However, the JumpROM wipe function does not take much time in either of these cases, so we recommend that you run this function in all cases.

2.3 PROTECTED RESOURCES

2.3.1 Protected Resources in Non-Volatile Storage

The bootloader uses AES keys stored in the key area of the non-volatile (EEPROM) file system to decrypt any encrypted segments of the file system. This key area is protected by algorithm

protection and must be cleared to 0xFFFF before the debug port can be externally unlocked. No other areas of the file system are protected.

For more information on the key area and encryption support, see the *Firmware Reference Manual* and Section 2.4, “File System Security Support” on page 5.

2.3.2 Protected Resources in Memory

You can protect data and code from external access to pre-defined sections of memory. When unlocking the debug port externally using the JumpROM functions, use the unlock command to clear these sections of memory before switching to unrestricted access mode. A list of protected memory resources is provided in the *Hardware Reference Manual* for your product.

2.4 FILE SYSTEM SECURITY SUPPORT

You can use encrypted data in the non-volatile memory to provide protection to software intellectual property for copyright and related purposes.

File system security is provided through the support of AES encrypted data, and as a result, a number of places in the firmware and program ROM support file system encryption. The components of this support are as follows:

- AES decryption implementation
- EEPROM file system
- Bootloader
- Non-volatile image creation tools

Documentation of some of these components has been provided in the appropriate sections of other manuals. Relevant points include:

- The peripherals access mode security blocks section in the *Hardware Reference Manual* outlines how security is provided, and presents an overview of the hardware components of this secure file system implementation.
- Chapter 4, “Debug Port Protocol” on page 11 of this manual provides a complete description of the restrictions placed on access through the debug port.
- The Program ROM function tables section provided in the *Firmware Reference Manual* provides links to the ROM-based implementations of the AES decryption functions.
- Program ROM support for AES and the decryption of encrypted file system components is documented in the file system decryption support section of the *Firmware Reference Manual*.
- The EEPROM file system security section and application encrypted sections section from the *Firmware Reference Manual* describe the EEPROM file system components.
- The Bootloader operation section of the *Firmware Reference Manual* describes the support in the bootloader for decryption (and the boot errors table lists the `BOOT_ERR_DECRYPT` error code that the bootloader lists in case of a decryption error).
- The Non-Volatile Memory Manager (NVMM) tool section in the *CFX Toolchain Reference* describes the support for creating encrypted (or partially encrypted) non-volatile images.

This page left blank for printing purposes.

3 JumpROM Function Set

3.1 BACKGROUND

The Program ROM contains functions that are executed through the debug port's JumpROM command. These functions provide access to information such as the Program ROM's version and error codes. Security functions for locking the device or wiping and unlocking the device are available. Functions are also provided to reset the system and to boot a selected application from an EEPROM attached to the device.

JumpROM functions all provide the same calling interface. This includes:

- A 16-bit argument passed as part of the JumpROM debug port command
- An optional 24-bit output stored to the debug port data register (P:0xE035)
- A software breakpoint that is executed when any JumpROM function completes

IMPORTANT: JumpROM functions do not maintain the current system state. As a result, when a JumpROM function finishes (with the exception of the JumpROM reset function), control passes to a wait loop in the JumpROM handler.

Details on the debug port protocol command set are provided in Section 4.3.6.6, "JumpROM Command" on page 30.

The following sections provide details on each of the functions in the JumpROM function set. A reference table listing the JumpROM commands is available in Appendix A, "JumpROM Function Quick Reference" on page 33.

3.2 JUMPROM NOP

Function Index	Input Description	Output Description
0x00	Not used	None

The JumpROM NOP function does not perform any function. It can be used to establish a connection to the external system.

3.3 JUMPROM GETVERSION

Function Index	Input Description	Output Description
0x01	Not used	Program ROM version number

The JumpROM GetVersion function can be used to check the Program ROM version. The version is returned in the debug register.

3.4 JUMPROM RESET

Function Index	Input Description	Output Description
0x02	Not used	None

The JumpROM Reset function resets the system. The reset is implemented through the watchdog timer. After the reset, the device starts its standard boot sequence and attempts to load the default application.

3.5 JUMPROM UNLOCK

Function Index	Input Description	Output Description
0x03	Not used	None

The JumpROM Unlock function attempts to unlock the debug port. The function first verifies that the protected resources in non-volatile storage are cleared, or that no EEPROM is attached. If the verification is successful, all protected resources in memory are cleared and the debug port is unlocked. For more information on file system security, see the *Firmware Reference Manual*.

3.6 JUMPROM WIPE

Function Index	Input Description	Output Description
0x04	AT_WREN opcode	Timeout status bit for EEPROM writes at bit position SR_X_ZERO (0 - No timeout, 1 - Timeout)

The JumpROM Wipe function attempts to clear the protected resources in non-volatile storage. The function first verifies that an EEPROM is attached. If an EEPROM is detected, it clears the key area, then automatically calls the JumpROM Unlock function. If an EEPROM is not detected, it clears the system memory and unlocks the system. For more information on file system security and algorithm protection schemes, see Section 2., “Algorithm Protection” on page 3 and the file system security section of the *Firmware Reference Manual*.

3.7 JUMPROM GETERROR

Function Index	Input Description	Output Description
0x06	Not used	System error code

The JumpROM GetError function is used to retrieve the latest system error code. The error code that is currently stored in the SYSVAR_SYS_ERRNO global system variable is returned in the debug register. The error codes are generally set during boot time. If the boot fails, more information about the failure can be learned if the error code is known. After a failed boot, the system waits several seconds before rebooting. During that time the error code can be retrieved.

Note: As with all JumpROM functions, executing this JumpROM function stops the system from rebooting because control is passed to the JumpROM handler.

See the *Firmware Reference Manual* for further information on Bootloader and BootROM error codes.

3.8 JUMPROM BOOTAPP

Function Index	Input Description	Output Description
0x07	Application to bootload	None

The JumpROM BootApp function is used to force the system to boot the specified application from EEPROM. The application is specified using its integer application number. The current running application (if any) is terminated. The boot sequence is executed, but instead of loading the default application, the parameter to JumpROM BootApp is used to specify which application to load.

If the parameter specifies an invalid application number, the boot will fail. The appropriate error code is generated and the system reboots after a delay. The subsequent boot will use the default application number from the file system.

For more information on application selection, see the *Firmware Reference Manual*.

IMPORTANT: The JumpROM BootApp function cannot be used when using an encrypted Bootloader (see the file system security section of the *Firmware Reference Manual*).

3.9 JUMPROM LOCK

Function Index	Input Description	Output Description
0x09	Not used	None

The JumpROM Lock function is used to unconditionally lock the debug port.

This page left blank for printing purposes.

4 Debug Port Protocol

4.1 INTRODUCTION

The debug port interface on the device allows a PC or other system supporting the two-wire Inter-IC (I^2C) protocol to control the CFX DSP externally. Information on the interface and hardware interface components is available in the *Hardware Reference Manual*.

The debug port interface implements a protocol layer, hereafter referred to as the *debug port protocol*, that provides the following functionality:

- Direct access to all CFX DSP core registers
- Direct access to all CFX DSP memory and memory-mapped control registers
- Run control, software, hardware, and external breakpoints, and single stepping capability
- JumpROM function set support

The debug port provides a user with only a peripheral view (through the contents of shared and microcode memory) of the HEAR processing of microcode configurations.

In this chapter, the PC or system communicating with the debug port is referred to as the *debug host*. Software running on the CFX DSP is referred to as the *DSP application*.

4.2 BASIC PROTOCOL INFORMATION

The debug port protocol uses an I^2C bus interface with an overlaid protocol command set to provide debug access to the CFX DSP core. The debug port acts as an I^2C slave device. The debug host must act as an I^2C master device.

See *The I^2C-Bus Specification* from Philips Semiconductors for more information.

4.2.1 Debug Port Addressing

The debug port responds to two I^2C addresses:

Internally-programmable
The DSP application can program the address internally through a register (`D_DEBUG_ADDRESS`).

Externally-programmable
You can program the address externally using the **Write Address** or **Acquire Address** commands.

Both of these addresses default to 0x60 on reset, and you can assign them any value between 0x00 to 0x7F.

For more information about I2C addressing, including typical pitfalls and addressing of the general-purpose I2C interface, see the *Hardware Reference Manual*.

4.2.2 Acknowledgements

The debug port protocol makes use of the I2C ACK and NAK mechanism for data acknowledgement with the master. After the master has sent the I2C Address and Read/Write bit, the debug port responds with an ACK by holding the data line low. In a Write transaction, the next byte that the master sends is the command byte. The response of an ACK or a NAK after the command byte depends on the byte received, the system state, and the debug port state.

If one or more of the following conditions is true, the debug port issues a NAK after the command byte:

- The debug port transmit state is *Initialize* and the debug host starts a read request. In this case, the debug port responds with a NAK immediately after the I2C Address and Read/Write bit.
- The debug port transmit state is *Initialize* and the debug host starts a write request where the command byte is not **Get Status Response**.
- The debug port receives an unknown command.
- The command is not valid in the current security mode (i.e., a command that is not available in restricted mode is attempted while access is restricted).
- The debug port is currently busy (depends on the command).
- The command is not valid in the current CFX DSP core run mode (e.g., attempting to read or write registers while the core is running).

After the debug port sends a NAK to the master, it ignores any further data from the master until a new I2C Start Condition is received.

4.2.3 Command Format

The debug host sends debug port protocol commands to the debug port in I2C write transfers. The debug port verifies proper transaction framing and format of the command. It also verifies that it is in the proper security and run mode to execute the command. The debug port then deals with executing the command by setting the appropriate states, data registers and control signals.

The general format of a debug port protocol command is shown below. Normal text indicates data sent from the debug host. Bold text indicates responses from the debug port.

`[S] [ADDR][W]` **`[A]`** `[CMDBYTE]` **`[A]`** `[BYTE0]` **`[A]`** `[BYTE1]` **`[A]`** `... [BYTEn]` **`[A]`** `[P]`

Where:

- `[S]` — I2C Start Condition
- `[ADDR]` — 7-bit debug port I2C Address
- `[W]` — Read/Write bit: 0 (Write) for commands
- `[CMDBYTE]` — Debug port protocol command byte
- **`[A]`** — Acknowledgement: ACK or NAK from debug port
- `[BYTEx]` — Byte x of the command sequence
- `[P]` — I2C Stop Condition

After some commands the debug port expects a fixed number of additional bytes. If the master sends more or fewer bytes than are expected before sending the Stop Condition, the debug port ignores the comand.

After other commands (**Write Memory**, **Write Normal Registers**, **Write Special Registers**, and **Set Hardware Breakpoints**), the debug port expect multiple blocks of data, such as memory word values or register values. For these commands, each complete block of data sent by the master is processed once it has been received. If the master sends a partial block of data before the Stop Condition, the debug port ignores the last block of data.

In the descriptions of commands in the following sections, the I^{2}C framing, addressing and acknowledgement data is omitted; the descriptions only show the command and data bytes.

4.2.4 Response Format

The debug port sends a debug port response after the debug host starts an I^{2}C read transfer. The general format of a response is shown below. Normal text indicates data sent from the debug host. Bold text indicates responses from the debug port.

`[S] [ADDR][R]` **`[A] [BYTE0]`** `[A]` **`[BYTE1]`**`[A] ...` **`[BYTEn]`**`[A] [P]`

Where:

- `[S]` — I^{2}C Start Condition
- `[ADDR]` — 7-bit debug port I^{2}C Address
- `[R]` — Read/Write bit: 1 (Read) for responses
- **`[A]`** — Acknowledgement: ACK or NAK from debug port
- **`[BYTEx]`** — Response byte X of the response sequence
- `[A]` — Acknowledgement: ACK or NAK from debug host
- `[P]` — I^{2}C Stop Condition

The contents of the response depend on the transmit state of the debug port, which is determined by the most recent command sent to the debug port.

In the descriptions of responses in the following sections, the I^{2}C framing, addressing and acknowledgement data is omitted; the descriptions only show the data bytes.

4.2.5 Modes

The debug port supports a number of independent modes that affect the execution of instructions on the CFX DSP, the contents of the debug port status response, and the response to debug port commands. These are summarized in Table 1.

TABLE 1: DEBUG PORT MODES

Mode	State	Description
CFX Run Mode	Running (default)	The CFX DSP is freely executing instructions.
	Stopped	The CFX DSP is not executing instructions because the core has been stopped (e.g., through an explicit stop, software breakpoint, hardware breakpoint, or external breakpoint). You can execute instructions by issuing single-step commands or by putting the debug port into Running mode.
Monitor Reset Mode	Disabled (default)	The Monitor Reset Bit (bit 12) of the debug port status response is 0.
	Enabled	The Monitor Reset BIt (bit 12) of the debug port status response is 1. Send the **Set Monitor Reset Bit** command to enable this mode. This mode is only disabled when the system is reset. You can use this mode for connection management.
Restricted Mode	Unrestricted Mode	The debug port executes all commands normally. The `ACCESS_RESTRICTED` bit is cleared in the `D_DEBUG_CFG` register.
	Restricted Mode (default)	The debug port NAKs some debug port commands, preventing unauthorized access to application code and data. The `ACCESS_RESTRICTED` bit is set in the `D_DEBUG_CFG` register.
Software Breakpoint Mode	Disabled (default)	The CFX DSP treats the `BREAKPOINT` instruction as a `NOP` instruction.
	Enabled	When the CFX DSP executes a `BREAKPOINT` instruction, the DSP is stopped and the status response indicates that a software breakpoint was hit. See Section 4.3.7, “Breakpoint Commands and Responses” on page 30 and the *CFX DSP Architecture Manual* for more information on software breakpoints.
Hardware Breakpoint N Mode (for N=0…7)	Disabled (default)	The CFX DSP does not stop at hardware breakpoint N.
	Enabled	After the CFX DSP executes the instruction at hardware breakpoint N, the DSP is stopped and the status response indicates that a hardware breakpoint was hit. The debug port supports eight hardware breakpoints that can be independently enabled and disabled. See Section 4.3.7, “Breakpoint Commands and Responses” on page 30 for more information on hardware breakpoints.
External Breakpoint Mode	Disabled (default)	The CFX DSP does not stop for external breakpoints.
	Enabled	When the external breakpoint condition is true, the CFX DSP is stopped and the status response indicates that an external breakpoint was hit. See Section 4.3.7, “Breakpoint Commands and Responses” on page 30 and the *Hardware Reference Manual* for more information on external breakpoints.

4.3 DEBUG PORT PROTOCOL COMMANDS AND RESPONSES

4.3.1 Overview

The debug port protocol consists of commands that are sent by the debug host to the debug port in I^2C Write transfers. The debug port decodes and executes these commands.

The description of each command lists the next transmit state. For commands that transfer data from the debug port to the debug host (**Read System ID**, **Read Memory**, **Read Normal Registers**, **Read Special Registers**, **Read Breakpoints**, **Read Debug Port Register** and **Read and Reset CRC**), the debug host must perform two I^2C transfers:

1. An I^2C write transfer containing the command and associated data—When the debug port executes the command, it changes its transmit state to the appropriate state.
2. An I^2C read transfer to transfer the data from the debug port to the debug host.

IMPORTANT: The debug port always returns to the *Status* transmit state after the next I^2C Read transfer.

The description of each command indicates the required security mode. Commands that specify "Any" for the security mode can be executed when the debug port is in either restricted or unrestricted mode. Commands that specify "Unrestricted" can only be executed when the debug port is in unrestricted mode. If the debug port receives a command and the debug port is not in the correct security mode, the debug port will not acknowledge the command.

Each description also lists the required run mode. The run mode indicates the required run mode for the CFX DSP core. To execute some commands, the core must be stopped; other commands can be executed while the core is either running or stopped. The debug port will not acknowledge a command when the run mode is incorrect; see Section 4.2.2, "Acknowledgements" on page 12 for more information.

The debug port protocol commands are summarized in Appendix B, "Debug Port Protocol Quick Reference" on page 35.

4.3.2 Initialize Transmit State

To guarantee that the debug host can detect when the debug port has reset, the debug port issues NAKs to all read requests until the debug host sends a **Read Status** command to the debug port. The debug port also does not acknowledge the command byte of all write requests other than a **Read Status** command. This is referred to as the *Initialize* transmit state.

The debug port changes to the *Initialize* transmit state when the debug port is reset. The debug port is reset when the system is reset for any reason, or the debug host sends the *Reset Debug Port* command.

4.3.3 Debug Port State and Information Commands and Responses

4.3.3.1 NOP Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x00	`[0x00]`	*Status*	Any	Any

Do nothing.

4.3.3.2 Read Status Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x53 ('S')	`[0x53]`	*Status*	Any	Any

Put the debug port in the *Status* transmit state. For more information, see Section 4.3.3.3, "Status Transmit State".

This must be the first command sent by the debug host after the debug port has reset. For more information, see Section 4.3.2, "Initialize Transmit State".

4.3.3.3 Status Transmit State

Syntax
`[STATUS 15:8] [STATUS 7:0] ... [STATUS 15:8] [STATUS 7:0]`

The debug port status response is the default response to most read requests from the debug host after establishing a connection. For example, sending a **NOP** command followed by a read from the slave returns the debug port status.

The status word consists of two bytes. The status response is repeated indefinitely as long as the master continues to acknowledge the response bytes. This allows the master to poll for a change in the status response using a single I^2C read transfer.

The status response bit fields are listed in Table 2.

TABLE 2: DEBUG PORT STATUS RESPONSE

Bit	Status Byte Fields	Values[1]
15	High Byte Indicator	1 – Indicates high byte of status response
14	Reserved	0
13	I^2C Bus Error Bit[2]	1 – Bus error has occurred 0 – No bus errors detected
12	Monitor Reset Bit	1 – **Set Monitor Reset Bit** command received since last system reset 0 – Default value (cleared by system reset only)
11	Security Mode	0 – Unrestricted mode 1 – Restricted mode
10	External Breakpoint Enable Status	0 – External breakpoints disabled 1 – External breakpoints enabled
9	Hardware Breakpoint Enable Status	0 – All hardware breakpoints disabled 1 – At least one hardware breakpoint enabled

TABLE 2: DEBUG PORT STATUS RESPONSE (CONTINUED)

Bit	Status Byte Fields	Values[1]
8	Software Breakpoint Enable Status	0 – Software breakpoints disabled 1 – Software breakpoints enabled
7	Low Byte Indicator	0 – Indicates low byte of status response
6	Reserved	0
5	Debug Port Busy	0 – Debug port idle 1 – Debug port busy[3]
4	External Breakpoint Hit Status	0 – External breakpoint not hit 1 – External breakpoint hit Cleared when the CFX DSP core is run or single-stepped.
3	Hardware Breakpoint Hit Status	0 – Hardware breakpoint not hit 1 – Hardware breakpoint hit Cleared when the CFX DSP core is run or single-stepped.
2	Software Breakpoint Hit Status	0 – Software breakpoint not hit 1 – Software breakpoint hit Cleared when the CFX DSP core is run or single-stepped.
1:0	Run state	00 – Running 01 – Stopped 10 – Core halted (SLEEP) 11 - Reserved

1. The bit changes between states when a change in the debug port state is observed unless otherwise specified.

2. The I²C Bus Error Bit indicates that an I²C watchdog timeout has occurred. This bit is cleared when the debug port executes the **Reset Debug Port** command, or when the DSP application reads the `D_I2C_ERROR_CNT` register. See the *Hardware Reference Manual* for more information.

3. The debug port is busy if the debug port is blocked from accessing memory due to a CFX and HEAR memory conflict. See the *Hardware Reference Manual* for more information.

4.3.3.4 Write Address Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x41 ('A')	`[0x41] [ADDRESS]`	*Status*	Any	Any

Set the externally-programmable debug port I²C address to the value specified in the argument. Bits 6:0 are used for the address. Bit 7 is ignored since I²C addresses are only 7 bits.

4.3.3.5 Acquire Address Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x04	[0x04] [MASK]	*Status*	Any	Any

Set the externally-programmable debug port I²C address based on the value of the five GPIO inputs and the value of the mask.

A mask is applied when acquiring the address. Bits 4:0 of the programmable address are the bit-wise AND of GPIO[4:0] and MASK[4:0]. Bits 6:5 of the programmable address are copies of MASK[6:5]. Bit 7 of the mask is ignored since I²C addresses are only 7 bits.

4.3.3.6 Reset Debug Port Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x07	[0x07]	*Initialize*	Any	Any

Reset the debug port. Resets the internally-programmable and externally-programmable debug port I²C addresses to their default values (0x60), clears the Monitor Reset Bit in the debug port status response, and changes to the *Initialize* transmit state.

4.3.3.7 Set Monitor Reset Bit Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x43 ('C')	[0x43]	*Status*	Any	Any

Set the Monitor Reset Bit in the debug port status response. The Monitor Reset Bit is only cleared by a system reset or the **Reset Debug Port** command.

4.3.3.8 Read System ID Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x49 ('I')	[0x49]	*SysID*	Any	Any

Put the debug port in the *SysID* transmit state so that the debug host can read the system identification information. For more information, see Section 4.3.3.9, "SysID Transmit State" on page 19.

4.3.3.9 SysID Transmit State

Syntax
[DEBUG PROTOCOL MAJOR] [DEBUG PROTOCOL MINOR] [CHIP FAMILY] [CHIP VERSION] [CHIP REVISION MAJOR] [CHIP REVISION MINOR]

The 6-byte system identification response contains information about the debug port protocol and the chip. Send the **Read System ID** command to set up this response.

This document describes debug port protocol version 1.9 ([DEBUG PROTOCOL MAJOR]=1, [DEBUG_PROTOCOL_MINOR]=9). If a future change to the debug protocol is backward-compatible with this version then the minor version number is updated. If a future change is not backward-compatible, the major version number is updated and the minor version number reset to 0.

The [CHIP FAMILY] is identified in the *Hardware Reference Manual* for your product.

The [CHIP VERSION], [CHIP REVISION MAJOR] and [CHIP REVISION MINOR] are combined to identify a particular version of a chip. Different chip versions are assigned to identify drastically different and incompatible chips within a chip family. A change to the major chip revision indicates that some functional change has been made to the chip that might affect DSP application code.

A change to the minor chip revision indicates that some internal functional change has been made to the chip that does not affect DSP application code.

Refer to the device identification section of the *Hardware Reference Manual* for more information about major and minor revisions.

4.3.3.10 Read and Reset CRC Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x4D ('M')	[0x4D]	*CRC*	Any	Any

Put the debug port in the *CRC* transmit state so that the debug host can read the debug port checksum. When the debug port executes this command, it stores the current CRC value to be sent to the debug host and resets the CRC value to 0xFFFF.

Use the **Read and Reset CRC** command to read the current CRC value and reset the CRC to 0xFFFF. The CRC includes all transferred bytes up to and including the **Read and Reset CRC** command. If the debug host reads the CRC using the next I^2C read transfer, the CRC bytes read by the debug host will be included in the next CRC calculation.

The debug port maintains a CRC of all bytes transmitted and received. The CRC is updated automatically after a complete byte has been transferred in either direction. This includes all bytes that are not acknowledged, invalid commands, or commands attempted in the incorrect security or run mode. If a byte is not completely transferred, it is not included in the CRC. The CRC does not include the address byte of I^2C transactions.

The debug port uses CRC-CCITT, which has the parameters described in Table 3.

TABLE 3: CRC-CCITT ALGORITHM PARAMETERS

CRC Parameter	Parameter Value
Order	16
Polynomial	$x^{16} + x^{12} + x^5 + 1$
Polynomial (hex)	0x1021
Initial Value (hex)	0xFFFF
Final XOR Value (hex)	0x0000[1]

1. No data manipulation is required (i.e., no data byte reversal, reversal of the final result or other finalization)

For more information, see Section 4.3.3.11, “CRC Transmit State”.

4.3.3.11 CRC Transmit State

Syntax
`[CRC 15:8] [CRC 7:0]`

Transfer the value of the 16-bit debug port CRC when the preceding **Read and Reset CRC** command was executed. For more information, see Section 4.3.3.10, “Read and Reset CRC Command” on page 19.

4.3.3.12 Reset and Start Cycle Counter Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x58 (‘X’)	`[0x58]`	*Status*	Unrestricted	Any

Reset the cycle count to 0 and start the cycle counter. For more information, see Section 4.3.3.14, “Cycle Count Transmit State”.

4.3.3.13 Get Cycle Count Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x56 (‘V’)	`[0x56]`	*Cycle Count*	Unrestricted	Any

Put the debug port in the *Cycle Count* transmit state so that the debug host can read the cycle count. For more information, see Section 4.3.3.14, “Cycle Count Transmit State”.

4.3.3.14 Cycle Count Transmit State

Syntax
[CYCLE COUNT 15:8] [CYCLE COUNT 7:0]

The debug port contains a cycle counter that counts CFX DSP core execution cycles while the CFX DSP core is running. When the chip is reset, the cycle count register is reset to 0 and does not count. To reset the cycle count register to 0 and start counting cycles, the debug host must send the **Reset and Start Cycle Counter** command. Use the **Get Cycle Count** command to retrieve the cycle count register's value. For more information about individual instruction execution cycles, see the *CFX DSP Architecture Manual*.

The cycle count register is 16 bits. When it reaches 0xFFFF, the cycle counter stops counting, and the register remains at 0xFFFF until it is reset.

4.3.4 Memory Access Commands and Responses

For memory transfers, the CFX DSP core must be stopped. The method by which the core was stopped does not matter (e.g., through an explicit stop, software breakpoint, hardware breakpoint or external breakpoint). If the core is not stopped, the commands to read or write memory will not be acknowledged.

4.3.4.1 Transfer Mode

The debug port memory access commands contain a Transfer Mode byte that specifies the memory space, data width, and transfer size for the read or write operation. The Transfer Mode byte is described in Table 4.

TABLE 4: TRANSFER MODE

Bit(s)	Description	Values
7:5	Reserved	000
4	Transfer Size	0 - Multiple words 1 - Single word
3:2	Memory Space	00 - Reserved 01 - X memory 10 - Y memory 11 - P memory
1:0	Data Width	00 - 8-bit 01 - 16-bit 10 - 24-bit 11 - 32-bit

Data is transferred with the most significant bytes first; for a 32-bit transfer, the sequence is:

1. Bits 31 to 24
2. Bits 23 to 16

3. Bits 15 to 8
4. Bits 7 to 0

The data width is specified independently from the memory space. If the transfer width is smaller than the actual memory width, the least significant bits are transferred. For example, for a 32-bit memory with an 8-bit transfer mode set, only bits 7 to 0 are transferred. When reading memory, the data is truncated from the actual memory width to the desired transfer width. When writing memory, the data is zero-extended from the transfer width to the actual memory width.

The debug port supports transferring either single words or multiple words. When the debug host is reading multiple words, the debug port queues up the next word to be sent, possibly triggering a side effect. For memory reads that cause side effects, the single word transfer can be used. In this mode, the debug port performs only a single access for the requested word.

When memory is being read in single word mode, subsequent bytes read from the debug port are all zero. Similarly, when memory is being written in single word mode, extra bytes received are ignored.

4.3.4.2 Read Memory Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x52 ('R')	`[0x52] [TRANSFER_MODE] [ADDR 15:8] [ADDR 7:0]`	*Memory*	Unrestricted	Stopped

Reading memory is initiated through the **Read Memory** debug port command.

The `[TRANSFER_MODE]` argument indicates the memory space and data width for the transfer, as described in Section 4.3.4.1, "Transfer Mode" on page 21. The next two bytes specify the high and low bytes of the starting address.

Once the Read Memory command has been sent, the debug port switches to the *Memory* transmit state.

To read the actual memory data, the debug host must start an I^2C read transfer. Depending on the specified data width, the address is automatically incremented after one, two, three, or four bytes have been transferred, and the subsequent word is queued to send.

The debug port performs no special processing on the address. The address automatically wraps around when it reaches the end of the address range (i.e., when the address reaches 0xFFFF, the next word read will be 0x0000).

Below is an example transaction for reading two words from X Memory (0x0010 and 0x0011) using the 24-bit transfer mode. Normal text indicates data sent from the debug host. Bold text indicates responses from the debug port.

```
[S] [ADDR][W] [A] ['R'] [A] [0x6] [A] [0x00] [A] [0x10] [A] [P]
[S] [ADDR][R] [A] [X(0010) 23:16] [A] [X(0010) 15:8] [A] [X(0010) 7:0] [A]
    [X(0011) 23:16] [A] [X(0011) 15:8] [A] [X(0011) 7:0] [A] [P]
```

4.3.4.3 Memory Transmit State

Syntax
`[DATA.0] [DATA.1] ... [DATA.n]`

Transfers data from the debug port to the debug host in response to a **Read Memory** command. For more information, see Section 4.3.4.2, “Read Memory Command”.

4.3.4.4 Write Memory Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode

0x57 (‘W’)	`[0x57] [TRANSFER_MODE]`
	`[ADDR 15:8] [ADDR 7:0]`
	`[DATA.0] [DATA.1] ... [DATA.n]`
		Status	Unrestricted	Stopped

Writing memory is initiated through the Write Memory debug port command.

The `[TRANSFER_MODE]` argument indicates the memory space and data width for the transfer as described in Section 4.3.4.1, “Transfer Mode” on page 21. The next two bytes specify the high and low bytes of the starting address.

The Write Memory transaction occurs in a single I^2C write transfer. After the debug port has acknowledged the memory address, the debug host can begin sending data to be written to memory. Depending on the transfer mode, the debug host sends one, two, three, or four bytes per word. Once the full word has been transmitted, the debug port queues the word for writing to memory and acknowledges the last byte. Hence the debug host can send data to the debug port continuously without the debug port having to stretch the clock. The debug port automatically increments the address by one after each write.

The debug port performs no special processing on the address. The address automatically wraps around when it reaches the end of the address range (i.e., when the address reaches 0xFFFF, the next word read is 0x0000).

Below is an example transaction for writing two words to X Memory (0xABCDEF to 0x0010 and 0x123456 to 0x0011) using the 24-bit transfer mode. Normal text indicates data sent from the debug host. Bold text indicates responses from the debug port.

`[S] [ADDR][W]` **`[A]`** `['W']` **`[A]`** `[0x6]` **`[A]`** `[0x00]` **`[A]`** `[0x10]` **`[A]`** `[0xAB]` **`[A]`** `[0xCD]` **`[A]`** `[0xEF]` **`[A]`**
`[0x12]` **`[A]`** `[0x34]` **`[A]`** `[0x56]` **`[A]`** `[P]`

4.3.5 Register Access Commands and Responses

The two categories of CFX DSP core registers are *normal* registers and *special* registers which have core requirements as follows:

- Before you access normal registers, the CFX DSP core must be stopped. The method by which the core is stopped does not matter (e.g., through an explicit stop, software breakpoint,

hardware breakpoint, or external breakpoint). If the core is not stopped, the commands to read or write normal registers will not be acknowledged.
- Special registers can be read when the CFX DSP core is running or stopped. The CFX DSP core must be stopped before writing to special registers.

The debug port also contains a special debug port hardware register, `D_DEBUG_DATA`. The DSP application can read or write this register from program memory at P:0xE035. The debug host can always read and write this register whether the CFX DSP core is running or stopped.

4.3.5.1 Read Normal Registers Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x44 ('D')	`[0x44] [REGISTER INDEX]`	*Normal Registers*	Unrestricted	Stopped

Put the debug port in the *Normal Registers* transmit state so that the debug host can read normal registers.

The `[REGISTER INDEX]` argument indicates the starting register index.

The response to this command is a sequence of register values. For each register, the debug port sends 24 bits, most significant byte first:

1. Bits 23 to 16
2. Bits 15 to 8
3. Bits 7 to 0

For registers that are smaller than the transfer width, the upper bits are zero padded. If a register is reserved or ignored, the debug port sends zero.

Once the debug port has sent a register, it increments the register index to transfer the next register. When the register index reaches 255, it wraps back to 0.

The normal register widths and indexes are listed in Table 5.

TABLE 5: NORMAL REGISTER INDEXES

Index	Register	Width		Index	Register	Width
0	Reserved			32	XCIRC0	15
1	Reserved			33	XCIRC1	15
2	Reserved			34	XCIRC2	15
3	Reserved			35	XCIRC3	15
4	X0	24		36	XCIRC4	15
5	X1	24		37	XCIRC5	15
6	Y0	24		38	XCIRC6	15
7	Y1	24		39	XCIFC7	15

TABLE 5: NORMAL REGISTER INDEXES (CONTINUED)

Index	Register	Width		Index	Register	Width
8	XP0	16		40	YCIRC0	15
9	XP1	16		41	YCIRC1	15
10	XP2	16		42	YCIRC2	15
11	XP3	16		43	YCIRC3	15
12	XP4	16		44	Ignored	
13	XP5	16		45	Ignored	
14	XP6	16		46	Ignored	
15	XP7	16		47	SP	16
16	XSTEP0	16		48	LR	16
17	XSTEP1	16		49	ILPC	16
18	XSTEP2	16		50	SR	16
19	XSTEP3	16		51	ILSR	16
20	XSTEP4	16		52	A0.L	24
21	XSTEP5	16		53	A1.L	24
22	XSTEP6	16		54	B0.L	24
23	XSTEP7	16		55	B1.L	24
24	YP0	16		56	A0.H	24
25	YP1	16		57	A1.H	24
26	YP2	16		58	B0.H	24
27	YP3	16		59	B1.H	24
28	YSTEP0	16		60	A0.E	8
29	YSTEP1	16		61	A1.E	8
30	YSTEP2	16		62	B0.E	8
31	YSTEP3	16		63	B1.E	8
				64-255	Reserved	

4.3.5.2 Normal Registers Transmit State

Syntax
[REG.0 23:16] [REG.0 15:8] [REG.0 7:0] ... [REG.n 23:16] [REG.n 15:8] [REG.n 7:0]

Transfer the values of normal registers. For more information, see Section 4.3.5.1, “Read Normal Registers Command” on page 24.

4.3.5.3 Write Normal Registers Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode

0x46 ('F')	`[0x46] [REGISTER INDEX]`
	`[REG.0 23:16] [REG.0 15:8]`
	`[REG.0 7:0] ... [REG.n 23:16]`
	`[REG.n 15:8] [REG.n 7:0]`
		Status	Unrestricted	Stopped

Write values to normal registers.

The `[REGISTER INDEX]` argument indicates the starting register index.

Subsequent bytes contain a sequence of register values. For each register, the debug host must send 24 bits, most significant byte first:

1. Bits 23 to 16
2. Bits 15 to 8
3. Bits 7 to 0

For registers that are smaller than the transfer width, the upper bits must be zero. If a register is Reserved or Ignored, the debug host must send a value of 0.

Once the debug port has received a register, it increments the register index to transfer the next register. When the register index reaches 255, it wraps back to 0.

The normal register widths and indexes are listed in Table 5 on page 24.

4.3.5.4 Read Special Registers Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x31 ('1')	`[0x31] [REGISTER INDEX]`	*Special Registers*	Unrestricted	Any

Put the debug port in the *Special Registers* transmit state so that the debug host can read special registers.

The `[REGISTER INDEX]` argument indicates the starting register index.

The response to this command is a sequence of register values. For each register, the debug port sends 16 bits, most significant byte first:

1. Bits 15 to 8
2. Bits 7 to 0

For registers that are smaller than the transfer width, the upper bits are zero padded. If a register is reserved or ignored, the debug port sends zero.

Once the debug port has sent a register, it increments the register index to transfer the next register. When the register index reaches 255, it wraps back to 0.

The special register widths and indexes are listed in Table 6.

TABLE 6: SPECIAL REGISTER INDEXES

Index	Register	Width		Index	Register	Width
0	LE0	16		8	LC0	12
1	LE1	16		9	LC1	12
2	LE2	16		10	LC2	12
3	LE3	16		11	LC3	12
4	LS0	16		12	LP	3
5	LS1	16		13	PC (Read-Only)	16
6	LS3	16		14-255	Reserved	
7	LS3	16				

The special registers LEn, LSn, LCn and LP represent the state of the CFX DSP core's hardware loop stack. LP is the loop pointer, and is 0 when no hardware loops are active, or 4, 5, 6 or 7 when 1, 2, 3 or 4 hardware loops are active respectively. LEn, LSn and LCn contain the values of LEA, LSA and LC respectively for each nested hardware loop. See the *CFX DSP Architecture Manual* for more information about hardware loops.

4.3.5.5 Special Registers Transmit State

Syntax
[REG.0 15:8] [REG.0 7:0] ... [REG.n 15:8] [REG.n 7:0]

Transfer the values of special registers. For more information, see Section 4.3.5.4, "Read Special Registers Command".

4.3.5.6 Write Special Registers Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x32 ('2')	[0x32] [REGISTER INDEX] [REG.0 15:8] [REG.0 7:0] ... [REG.n 15:8] [REG.n 7:0]	*Status*	Unrestricted	Stopped

Write values to special registers.

The [REGISTER INDEX] argument indicates the starting register index.

Subsequent bytes contain a sequence of register values. For each register, the debug host must send 16 bits, most significant byte first:

1. Bits 15 to 8
2. Bits 7 to 0

For registers that are smaller than the transfer width, the upper bits must be zero. If a register is Reserved, the debug host must send a value of 0.

Once the debug port has received a register, it increments the register index to transfer the next register. When the register index reaches 255, it wraps back to 0.

The special register widths and indexes are listed in Table 6 on page 27.

You cannot use this command to modify the program counter (PC). Instead, you must single-step the GOTO.0D instruction using the **Execute Instruction** command. See Section 4.3.6.5, "Execute Instruction Command" on page 30 for more information.

4.3.5.7 Read Debug Port Register Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x33 ('3')	[0x33]	*Debug Port Register*	Any	Any

Put the debug port in the *Debug Port Register* transmit state so that the debug host can read the debug port register.

The DSP application can read and write the debug port register, D_DEBUG_DATA. The debug host can read this register regardless of run mode or security mode.

4.3.5.8 Debug Port Register Transmit State

Syntax
[DATA 31:24] [DATA 23:16] [DATA 15:8] [DATA 7:0]

Transfer the value of the debug port register. For more information, see Section 4.3.5.7, "Read Debug Port Register Command".

4.3.5.9 Write Debug Port Register Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x34 ('4')	[0x34] [DATA 31:24] [DATA 23:16] [DATA 15:8] [DATA 7:0]	*Status*	Any	Any

Write a value to the debug port register, D_DEBUG_DATA. This register can be read and written by the DSP application, and the debug host can write to this register regardless of run mode or security mode.

4.3.6 Run Control Commands

4.3.6.1 Start Core Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x47 ('G')	`[0x47]`	*Status*	Any	Any

Puts the debug port into Running mode, allowing the CFX DSP core to run freely.

4.3.6.2 Stop Core Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x50 ('P')	`[0x50]`	*Status*	Any	Any

Puts the debug port in Stopped mode, stopping the CFX DSP core.

4.3.6.3 Single-Step Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x54 ('T')	`[0x54]`	*Status*	Unrestricted	Stopped

Execute an instruction from the DSP application on the CFX DSP core. When the debug port executes this command, the CFX DSP core ignores any pending enabled interrupts.

For instructions with delay slots, this command will execute all instructions in the delay slots.

4.3.6.4 Single-Step With Interrupts Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x4E ('N')	`[0x4E]`	*Status*	Unrestricted	Stopped

Execute an instruction from the DSP application on the CFX DSP core. If interrupts are enabled on the CFX core and there is a pending enabled interrupt when the debug port executes this command, the CFX DSP core will execute the interrupt vector operation instead of the instruction at the program counter. After the vector operation, the program counter points to the first instruction of the interrupt handler.

For instructions with delay slots, this command will execute all instructions in the delay slots.

4.3.6.5 Execute Instruction Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x4F ('O')	`[0x4F] [INSTR 31:24] [INSTR 23:16] [INSTR 15:8] [INSTR 7:0]`	*Status*	Unrestricted	Stopped

Execute the instruction provided on the CFX DSP core. This command does not modify the program counter (`PC`) unless the instruction modifies `PC`.

4.3.6.6 JumpROM Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x4A ('J')	`[0x4A] [CMD 7:0] [ARG 15:8] [ARG 7:0]`	*Status*	Any	Any

This command causes the CFX DSP core to execute the JumpROM function specified. For more information regarding the various JumpROM functions and their functionality, see Chapter 3, "JumpROM Function Set" on page 7.

When the debug port receives this command, it transfers control of the CFX DSP core to the JumpROM function. When the JumpROM function completes, the CFX DSP executes a `BREAKPOINT` instruction within an infinite loop. To execute a JumpROM function, we recommend that the debug host follow these steps:

1. Establish a debug port connection.
2. Stop the CFX DSP core.
3. Send the **Write Breakpoint Config** command to enable software breakpoints.
4. Send the **JumpROM** command to transfer control to the JumpROM function.
5. Poll the status response to detect when the CFX DSP core executes the `BREAKPOINT` instruction.
6. Read the response to the JumpROM command, if any, from the debug port register.

4.3.7 Breakpoint Commands and Responses

4.3.7.1 Set Hardware Breakpoints Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x42 ('B')	`[0x42] [HWBP.0] [ADDR.0 15:8] [ADDR.0 7:0] [HWBP.1] [ADDR.1 15:8] [ADDR.1 7:0] ... [HWBP.n] [ADDR.n 15:8] [ADDR.n 7:0]`	*Status*	Unrestricted	Any

The debug host sends this command to configure hardware breakpoints. Each hardware breakpoint can be individually enabled or disabled. A hardware breakpoint causes the CFX DSP core to stop when it executes the instruction at a specified address.

The debug port supports up to 8 hardware breakpoints. The debug host must send three bytes to configure each hardware breakpoint. The `[HWBP]` byte selects the desired hardware breakpoint and indicates whether it should be enabled or disabled, as described in Table 7. The following two bytes specify the hardware breakpoint address.

TABLE 7: HWBP.N FIELDS

Bit(s)	Description	Values
7	Enable/Disable	0 - Disable the hardware breakpoint 1 - Enable the hardware breakpoint
6:3	Reserved	0
2:0	Hardware breakpoint index	0-7

The debug host can configure hardware breakpoints in any order. The debug port configures each hardware breakpoint immediately when it receives the `[HWBP]` byte and two address bytes for that breakpoint.

4.3.7.2 Write Breakpoint Config Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x4B ('K')	`[0x4B] [ESWBP] [HWBP]`	*Status*	Any	Any

Send this command to enable and disable software, hardware and external breakpoints. Each type of breakpoint and each individual hardware breakpoint can be enabled or disabled separately. The `[ESWBP]` byte configures software and external breakpoints, as described in Table 8. The `[HWBP]` byte configures hardware breakpoints, as described in Table 9.

TABLE 8: ESWBP FIELDS

Bit(s)	Description	Values
7:2	Reserved	0
1	External breakpoints	0 - Disable external breakpoints 1 - Enable external breakpoints
0	Software breakpoints	0 - Disable software breakpoints 1 - Enable software breakpoints

TABLE 9: HWBP FIELDS

Bit(s)	Description	Values
7	Enable/Disable 7	0 - Disable hardware breakpoint 7 1 - Enable hardware breakpoint 7
…		
0	Enable/Disable 0	0 - Disable hardware breakpoint 0 1 - Enable hardware breakpoint 0

4.3.7.3 Read Breakpoints Command

Command Byte	Syntax	Transmit State	Security Mode	CFX Run Mode
0x45 ('E')	`[0x45]`	*Breakpoints*	Unrestricted	Any

Put the debug port in the *Breakpoints* transmit state so that the debug host can read the current breakpoint configuration.

4.3.7.4 Breakpoints Transmit State

Syntax
`[ESWBP] [HWBP] [ADDR.0 15:8] [ADDR.0 7:0] ... [ADDR.7 15:8] [ADDR.7 7:0]`

Transfer the current breakpoint configuration in response to the **Read Breakpoints** command.

The `[ESWBP]` and `[HWBP]` bytes indicate whether software, hardware, and external breakpoints are enabled. These bytes are described in Section 4.3.7.2, "Write Breakpoint Config Command" on page 31. The remaining bytes indicate the configured breakpoint addresses for hardware breakpoints 0-7 in ascending order.

A JumpROM Function Quick Reference

TABLE 10: JUMPROM FUNCTIONS

Function Index	JumpROM Function	Input	Output	Description
0x00	NOP	Not used	None	No operation
0x01	GetVersion	Not used	Program ROM version number	Load the Program ROM version into the debug port data register
0x02	Reset	Not used	None	Reset the system
0x03	Unlock	Not used	None	Verify secure EEPROM area is cleared, then clear memory and unlock the device
0x04	Wipe	AT_WREN opcode	Timeout status bit for EEPROM writes at bit position SR_X_ZERO (0 - No timeout, 1 - Timeout)	Clear the secure EEPROM area, then clear memory and unlock the device
0x05	-			Reserved
0x06	GetError	Not used	System error code	Load the current error number into the debug port data register
0x07	BootApp	Application to bootload	None	Boot the specified application
0x08	-			Reserved
0x09	Lock	Not used	None	Lock the device

This page left blank for printing purposes.

B Debug Port Protocol Quick Reference

TABLE 11: DEBUG PORT PROTOCOL COMMANDS

Command	Command Byte	Syntax and Description	Transmit State	Security Mode	CFX Run Mode
NOP	0x00	`[0x00]`	*Status*	Any	Any
Read Status	0x53 ('S')	`[0x53]`	*Status*	Any	Any
Write Address	0x41 ('A')	`[0x41] [ADDRESS]`	*Status*	Any	Any
Acquire Address	0x04	`[0x04] [MASK]`	*Status*	Any	Any
Reset Debug Port	0x07	`[0x07]`	*Initialize*	Any	Any
Set Monitor Reset Bit	0x43 ('C')	`[0x43]`	*Status*	Any	Any
Read System ID	0x49 ('I')	`[0x49]`	*SysID*	Any	Any
Read and Reset CRC	0x4D ('M')	`[0x4D]`	*CRC*	Any	Any
Reset and Start Cycle Counter	0x58 ('X')	`[0x58]`	*Status*	Unrestricted	Any
Get Cycle Count	0x56 ('V')	`[0x56]`	*Cycle Count*	Unrestricted	Any
Read Memory	0x52 ('R')	`[0x52] [TRANSFER_MODE]` `[ADDR 15:8] [ADDR 7:0]`	*Memory*	Unrestricted	Stopped

Write Memory	0x57 ('W')	`[0x57] [TRANSFER_MODE]`
		`[ADDR 15:8] [ADDR 7:0]`
		`[DATA.0] [DATA.1] ... [DATA.n]`
			Status	Unrestricted	Stopped

Read Normal Registers	0x44 ('D')	`[0x44] [REGISTER INDEX]`	*Normal Registers*	Unrestricted	Stopped

Write Normal Registers	0x46 ('F')	`[0x46] [REGISTER INDEX]`
		`[REG.0 23:16] [REG.0 15:8]`
		`[REG.0 7:0] ... [REG.n 23:16]`
		`[REG.n 15:8] [REG.n 7:0]`
			Status	Unrestricted	Stopped

Read Special Registers	0x31 ('1')	`[0x31] [REGISTER INDEX]`	*Special Registers*	Unrestricted	Any

Write Special Registers	0x32 ('2')	`[0x32] [REGISTER INDEX]`
		`[REG.0 15:8] [REG.0 7:0] ...`
		`[REG.n 15:8] [REG.n 7:0]`
			Status	Unrestricted	Stopped

Read Debug Port Register	0x33 ('3')	`[0x33]`	*Debug Port Register*	Any	Any

TABLE 11: DEBUG PORT PROTOCOL COMMANDS (CONTINUED)

Command	Command Byte	Syntax and Description	Transmit State	Security Mode	CFX Run Mode
Write Debug Port Register	0x34 ('4')	[0x34] [DATA 31:24] [DATA 23:16] [DATA 15:8] [DATA 7:0]	*Status*	Any	Any
Start core	0x47 ('G')	[0x47]	*Status*	Any	Any
Stop core	0x50 ('P')	[0x50]	*Status*	Any	Any
Single-step	0x54 ('T')	[0x54]	*Status*	Unrestricted	Stopped
Single-step with Interrupts	0x4E ('N')	[0x4E]	*Status*	Unrestricted	Stopped
Execute Instruction	0x4F ('O')	[0x4F] [INSTR 31:24] [INSTR 23:16] [INSTR 15:8] [INSTR 7:0]	*Status*	Unrestricted	Stopped
JumpROM	0x4A ('J')	[0x4A] [CMD 7:9] [ARG 15:8] [ARG 7:0]	*Status*	Any	Any
Set Hardware Breakpoints	0x42 ('B')	[0x42] [HWBP.0] [ADDR.0 15:8] [ADDR.0 7:0] ... [HWBP.n] [ADDR.n 15:8] [ADDR.n 7:0] where: HWBP.n[7] – Enable (1)/Disable (0) BP HWBP.n[6:3] – Reserved HWBP.n[2:0] - Hardware BP Index (0-7) ADDR.n – Address for breakpoint	*Status*	Unrestricted	Any
Write Breakpoint Config	0x4B ('K')	[0x4B] [ESWBP] [HWBP] where: ESWBP[0] – Enable(1)/Disable(0) SW BPs ESWBP[1] – Enable(1)/Disable(0) Ext. BPs HWBP[0] - Enable(1)/Disable(0) HW BP 0 HWBP[1] - Enable(1)/Disable(0) HW BP 1 ... HWBP[7] - Enable(1)/Disable(0) HW BP 7	*Status*	Any	Any
Read Breakpoints	0x45 ('E')	[0x45]	*Breakpoints*	Unrestricted	Any

TABLE 12: DEBUG PORT PROTOCOL TRANSMIT STATES

Transmit State	Syntax
Status	[STATUS 15:8] [STATUS 7:0] ... [STATUS 15:8] [STATUS 7:0]
SysID	[DEBUG PROTOCOL MAJOR] [DEBUG PROTOCOL MINOR] [CHIP FAMILY] [CHIP VERSION] [CHIP REVISION MAJOR] [CHIP REVISION MINOR]
CRC	[CRC 15:8] [CRC 7:0]

TABLE 12: DEBUG PORT PROTOCOL TRANSMIT STATES (CONTINUED)

Transmit State	Syntax
Cycle Count	`[CYCLE COUNT 15:8] [CYCLE COUNT 7:0]`
Memory	`[DATA.0] [DATA.1] ... [DATA.n]`
Normal Registers	`[REG.0 23:16] [REG.0 15:8] [REG.0 7:0] ...` `[REG.n 23:16] [REG.n 15:8] [REG.n 7:0]`
Special Registers	`[REG.0 15:8] [REG.0 7:0] ... [REG.n 15:8] [REG.n 7:0]`
Debug Port Register	`[DATA 31:24] [DATA 23:16] [DATA 15:8] [DATA 7:0]`
Breakpoints	`[ESWBP] [HWBP] [ADDR.0 15:8] [ADDR.0 7:0] ... [ADDR.7 15:8] [ADDR.7 7:0]`

TABLE 13: DEBUG PORT STATUS RESPONSE

Bit	Status Byte Fields	Values[1]
15	High Byte Indicator	1 – Indicates high byte of status response
14	Reserved	0
13	I^2C Bus Error Bit[2]	1 – Bus error has occurred 0 – No bus errors detected
12	Monitor Reset Bit	1 – **Set Monitor Reset Bit** command received since last system reset 0 – Default value (cleared by system reset only)
11	Security Mode	0 – Unrestricted mode 1 – Restricted mode
10	External Breakpoint Enable Status	0 – External breakpoints disabled 1 – External breakpoints enabled
9	Hardware Breakpoint Enable Status	0 – All hardware breakpoints disabled 1 – At least one hardware breakpoint enabled
8	Software Breakpoint Enable Status	0 – Software breakpoints disabled 1 – Software breakpoints enabled
7	Low Byte Indicator	0 – Indicates low byte of status response
6	Reserved	0
5	Debug Port Busy	0 – Debug port idle 1 – Debug port busy[3]
4	External Breakpoint Hit Status	0 – External breakpoint not hit 1 – External breakpoint hit Cleared when the CFX DSP core is run or single-stepped.

TABLE 13: DEBUG PORT STATUS RESPONSE (CONTINUED)

Bit	Status Byte Fields	Values[1]
3	Hardware Breakpoint Hit Status	0 – Hardware breakpoint not hit 1 – Hardware breakpoint hit Cleared when the CFX DSP core is run or single-stepped.
2	Software Breakpoint Hit Status	0 – Software breakpoint not hit 1 – Software breakpoint hit Cleared when the CFX DSP core is run or single-stepped.
1:0	Run state	00 – Running 01 – Stopped 10 – Core halted (SLEEP) 11 - Reserved

1. The bit changes between states when a change in the debug port state is observed unless otherwise specified.

2. The I^2C Bus Error Bit indicates that an I^2C watchdog timeout has occurred. This bit is cleared when the debug port executes the **Reset Debug Port** command, or when the DSP application reads the `D_I2C_ERROR_CNT` register. See the *Hardware Reference Manual* for more information.

3. The debug port is busy if the debug port is blocked from accessing memory due to a CFX and HEAR memory conflict. See the *Hardware Reference Manual* for more information.

TABLE 14: TRANSFER MODE

Bit(s)	Description	Values
7:5	Reserved	000
4	Transfer Size	0 - Multiple words 1 - Single word
3:2	Memory Space	00 - Reserved 01 - X memory 10 - Y memory 11 - P memory
1:0	Data Width	00 - 8-bit 01 - 16-bit 10 - 24-bit 11 - 32-bit

TABLE 15: NORMAL REGISTER INDEXES

Index	Register	Width		Index	Register	Width
0	Reserved			32	`XCIRC0`	15
1	Reserved			33	`XCIRC1`	15

TABLE 15: NORMAL REGISTER INDEXES (CONTINUED)

Index	Register	Width		Index	Register	Width
2	Reserved			34	XCIRC2	15
3	Reserved			35	XCIRC3	15
4	X0	24		36	XCIRC4	15
5	X1	24		37	XCIRC5	15
6	Y0	24		38	XCIRC6	15
7	Y1	24		39	XCIFC7	15
8	XP0	16		40	YCIRC0	15
9	XP1	16		41	YCIRC1	15
10	XP2	16		42	YCIRC2	15
11	XP3	16		43	YCIRC3	15
12	XP4	16		44	Ignored	
13	XP5	16		45	Ignored	
14	XP6	16		46	Ignored	
15	XP7	16		47	SP	16
16	XSTEP0	16		48	LR	16
17	XSTEP1	16		49	ILPC	16
18	XSTEP2	16		50	SR	16
19	XSTEP3	16		51	ILSR	16
20	XSTEP4	16		52	A0.L	24
21	XSTEP5	16		53	A1.L	24
22	XSTEP6	16		54	B0.L	24
23	XSTEP7	16		55	B1.L	24
24	YP0	16		56	A0.H	24
25	YP1	16		57	A1.H	24
26	YP2	16		58	B0.H	24
27	YP3	16		59	B1.H	24
28	YSTEP0	16		60	A0.E	8
29	YSTEP1	16		61	A1.E	8
30	YSTEP2	16		62	B0.E	8
31	YSTEP3	16		63	B1.E	8
				64-255	Reserved	

TABLE 16: SPECIAL REGISTER INDEXES

Index	Register	Width	Index	Register	Width
0	LE0	16	8	LC0	12
1	LE1	16	9	LC1	12
2	LE2	16	10	LC2	12
3	LE3	16	11	LC3	12
4	LS0	16	12	LP	3
5	LS1	16	13	PC (Read-Only)	16
6	LS3	16	14-255	Reserved	
7	LS3	16			

This page left blank for printing purposes.

Global Headquarters
ON Semiconductor
5005 East McDowell Road
Phoenix, Arizona 85008 USA
Tel: +1 602 244 6600

Website
www.onsemi.com

Copyright © 2011 Semiconductor Components Industries, LLC d/b/a ON Semiconductor. ON Semiconductor and the ON logo are registered trademarks of Semiconductor Components Industries, LLC (ON Semiconductor). All other brands, product names and company names mentioned herein may be trademarks or registered trademarks of their respective holders.